

Project KT — Manuscript → Illustrated Picture-Book Pipeline

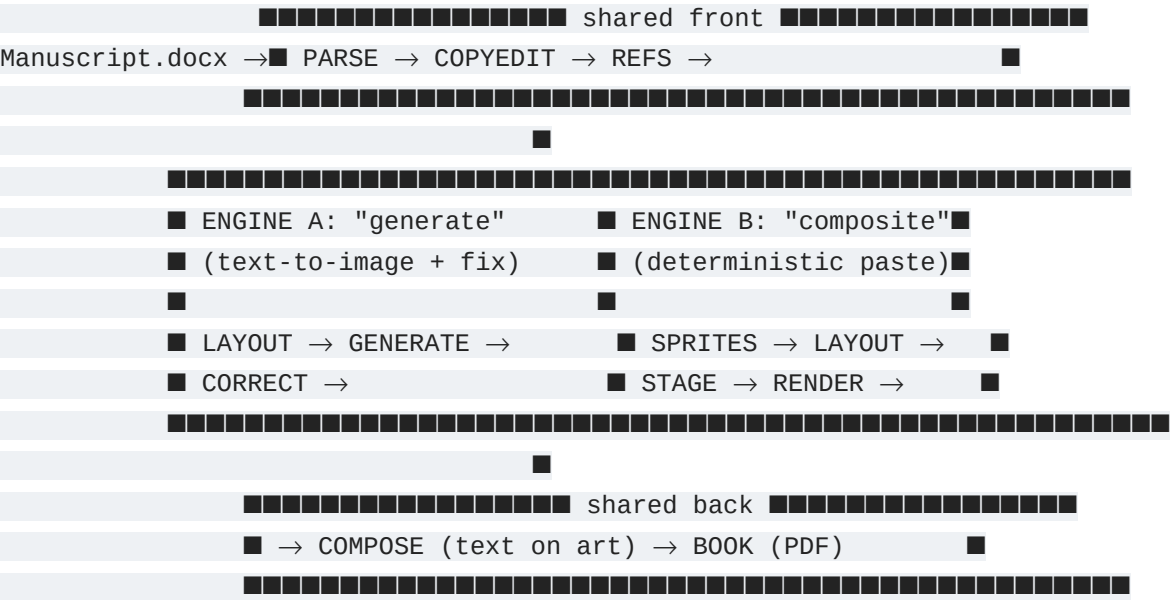
Purpose of this document: a complete knowledge-transfer for an AI-image-generation expert joining to help us get to *perfect* illustrations. It explains, in plain language: the goal, the end-to-end flow, what TOON is and how we generate it, what every code stage does, the two rendering engines we built, the exact models we call, the full history of what we tried and why, where it still fails, and where we think the real fix (LoRA) lives.

1. What the project does (in one paragraph)

We take a children’s book **manuscript** (a `.docx` file) and turn it into a **finished, print-ready illustrated PDF** — fully automatically. A text model reads the story and acts as art director + character designer; image models draw the pages; code lays out the type and assembles the book. The single hardest problem, and the client’s #1 complaint, is **character consistency**: the same dog / girl / parent must look *identical* on every page (same face, coat, cap, size), while the scene around them changes. Almost every version of this project has been an attempt to solve that one problem.

2. The big-picture flow

There is a single orchestrator, `pipeline/run_v3.py`, that runs the stages in order. It is **resumable** (`--from <stage>`) and can be limited to specific pages (`--only 4,11`). There are **two engines** that share the same front and back stages but differ in how they draw the characters:



- **Engine A — generate:** draw each page from scratch with a text-to-image model, then run an AI “corrector” that compares each page to a reference sheet and edits out the mistakes. Flexible, beautiful, but the characters **drift** page to page.

- **Engine B — composite** (current default): draw each character **once** as a set of cut-out “sprites,” then **paste the exact same pixels** onto every page and blend them in. Characters are pixel-identical by construction; only the background varies.

Both engines are real and in the repo. Composite is the current default because it structurally guarantees the consistency that Engine A can only approximate.

3. The two AI models we call (everything goes through OpenRouter)

We do **not** host any model. All calls go to **OpenRouter’s** chat-completions API.

Role	Model ID	Used for	Wrapper
Text + Vision	anthropic/claude-sonnet	Reading the manuscript, designing characters, planning scenes, copyediting, <i>judging</i> generated art (vision)	pipeline/llm.py
Image editing	google/gemini-3-pro-image	Image/reference sheets, page art, and in-place edits that preserve everything unmentioned	pipeline/editor.py

Two important properties we lean on:

- **Gemini image is an *editor*, not just a generator.** Given an image + an instruction, it changes only what you asked (e.g. “add the green cap”) and keeps the rest of the frame. This is what makes the corrector and the compositor’s “harmonize” step possible.
- **Gemini defaults to landscape (~1.83:1).** A square book would crop that badly, so we always pass a **blank white square as the edit base** (`editor.blank_square()`) to force 1:1 output, and centre-crop any stray landscape as a safety net (`editor.to_square()`).

Both wrappers have a **hard-deadline** watchdog: a single slow/hung connection once froze a whole book run for 95 minutes, so each request now runs in a worker thread that is abandoned if it overruns (~240s), feeding a normal retry loop.

4. TOON — what it is and how we generate it

TOON = Token-Oriented Object Notation. The client asked us to use it. It encodes uniform arrays as a compact **tabular block** (write the header once, then rows) instead of JSON's repeated keys and braces. The payoff is **fewer input tokens** when we inject this data into LLM prompts — and we inject it a lot.

How we actually store it (an important nuance): the Python `python-toon` encoder is reliable, but its *decoder* trips on our deeply-nested data (scenes contain a list of characters; groups contain a list of members). So `pipeline/toon_io.py` does this:

- **On disk, the canonical file is JSON** (100% reliable round-trip) — even though the file is named `*.toon`.
- A human-readable `.view.toon` sidecar is written alongside for inspection and to satisfy the client's requested format.
- When we inject data into a prompt, `for_prompt()` renders the compact TOON — which is exactly where the token savings matter.
- Tabular arrays use a **tab delimiter**, because prose values are full of commas.

How TOON is generated — this is the PARSE stage (`pipeline/parse_v3.py`):

1. Read every paragraph out of the manuscript `.docx`.
2. Send the whole text to Claude with a big “**you are a children's book art director and character designer**” system prompt.
3. Claude returns one JSON art-plan, which we split into two files: - `characters.toon` → { `style`, `characters[]`, `lookalike_groups[]` } - `scenes.toon` → { `title`, `author`, `settings[]`, `scenes[]` }

What makes this plan powerful (richer than a hand-written “story bible”):

- **One cohesive art style** chosen to fit *this* book (medium, linework, palette, lighting, mood, influences).
- **A locked_spec for every recurring (“high-consistency”) character** — an exact, enumerated appearance where *every* trait has ONE committed value (no “X or Y”, no “often”), colours are **hex codes**, and any gap the manuscript leaves is *invented and locked*. This is the anti-drift backbone (see §6).
- **Look-alike groups** — characters that could be confused (e.g. two golden retrievers) plus the single feature that tells them apart.
- **Settings** — recurring locations described exactly enough to redraw the *same* place every time.
- **Per-page scenes** — setting, who's in it, action, mood, camera, and where the caption text should sit. The page **text is copied verbatim** from the manuscript.

Two guardrails enforced here: **MAN-1** (page text must be verbatim — never paraphrased or invented) and **MAN-2** (don't invent or drop pages). A `reconcile()` check flags any body text that isn't found word-for-word in the manuscript.

5. Every stage, in plain language

Shared front stages

PARSE (`parse_v3.py`) — manuscript → `characters.toon` + `scenes.toon`. (*Claude*) Described in §4. The art-director + character-designer step.

COPYEDIT (`copyedit_v3.py`) — corrects each page's prose to the **Ballast house style guide** (numbers, dates, ellipses `. . .`, dashes, italics, a canonical word list). (*Claude, temperature 0.*) Runs deterministic fixes first, then the model; refuses any edit that changes the line count (so pagination stays intact); writes an auditable `copyedit_report.md`. Skippable with `--no-copyedit`. **Note:** this is a *prose* standard, not art direction.

REFS (`refs_v3.py`) — draws a clean **reference sheet** (portrait + full body) for every high-consistency character, plus a **combined “duo/group” sheet** for look-alikes so the distinguishing feature is anchored in one image the model can't average away. (*Gemini image.*) Sheets are generated **square** (same reasons as §3). For real people, it can condition on an uploaded photo for likeness. These sheets are the visual ground-truth every later stage compares against.

Engine A — generate (text-to-image + AI correction)

LAYOUT (`layout_v3.py`) — Claude assigns each character a normalized bounding box and picks which edge (top/bottom) stays calm for the caption.

GENERATE (`generate_v3.py`) — the text-to-image page draw. (*Gemini image.*) Builds a detailed prompt = scene + the exact character locks + the reference sheets

hard rules from the checklist (§7). Key tricks: forces **one continuous scene** (explicitly *not* a grid/collage of panels); composes with a high or low horizon so ~38% of the frame is genuinely open sky or ground for the caption; and **measures the “energy” of that reserved band**, regenerating if it isn't actually calm.

CORRECT (`correct_v3.py`) — the QA + repair loop. Two separate **vision judges** (*Claude vision*) look at each page: 1. an **identity judge** compares each character to its reference sheet and flags drift (wrong hair, missing cap, wrong coat); 2. a **composition judge** catches things the identity judge misses — inconsistent character **scale**, a figure “sitting on air” (ungrounded), legs-only/headless crops, warped anatomy, split panels. Flagged problems become an **in-place edit instruction** to Gemini, which fixes only the named character while preserving the scene. Up to 2 passes; originals backed up as `.orig.png`.

Engine B — composite (deterministic, current default)

SPRITES (`sprites_v3.py`) — draw a small **atlas** of poses (sit, walk, run...) per character **once**, from its reference sheet. (*Gemini image.*) Background is cut away with `rembg` to transparent PNGs. This is the pixel library reused everywhere.

STAGE (`stage_v3.py`) — the “art director without a pen.” (*Claude.*) For each page it decides **which pose, position (x,y), height, and facing** each character takes, and which edge stays calm for the

caption. Same-type characters get equal height (so two dogs match). Output: `staging.toon`.

RENDER (`render_v3.py`) — assembles the page in deterministic steps: (1) paint a **character-free background plate** locked to the location reference (*Gemini*), (2) **paste the atlas sprites** at the staged size/position (`composite_v3.py`: scale, anchor feet, soft shadow, feathered edges — pure PIL, no AI), (3) **harmonize** the whole thing through a Gemini `img2img` pass told to keep appearance and size *exactly* — this removes the “pasted-on” look that killed our first compositing attempt, (4) record each character’s bounding box so text can avoid them.

Shared back stages

COMPOSE (`compose_v3.py`) — places the caption **on the art**, in genuine negative space, with **no card or border**. Locked type size (~3% of page height, so every page matches), a hard safe margin so text never clips the edge, a soft white halo for legibility on any background, and adaptive ink (dark on light / light on dark). It slides a text-height strip down the page and picks the calmest spot (a vision call is only a fallback). Front matter is special-cased: **cover/title** get the book title in a display serif; **dedication** is centred verbatim.

BOOK (`book_v3.py`) — normalizes every page to one uniform square trim (**2550 px = 8.5” @ 300 dpi**, cover-crop so nothing drifts) and writes the final **PDF** in reading order. Pure code, no AI.

Shared logic / support (no model calls)

- **plan_v3.py** — loads the plan and exposes the helpers every stage uses; crucially `char_lock()` returns the canonical identity text (preferring the exact `locked_spec`). This one chokepoint is why *all* stages speak the same character description.
- **charspec.py** — serializes a `locked_spec` into **deterministic** text (same spec in → identical text out). Emphatically re-states left/right for wrist items (models flip them), puts headwear first (signature caps must never drop), and spells out “PLAIN chest, absolutely NO logo” so the model doesn’t invent one.
- **checklist_v3.py** — the single source of truth for all client feedback, rendered as rule blocks injected into generate, correct, and the judge (§7).
- **llm.py / editor.py** — the OpenRouter clients (§3).
- **style_guide.py** — loads the Ballast prose guide and appends it to text prompts.
- **toon_io.py** — the TOON/JSON I/O layer (§4).

6. The consistency backbone: the “locked spec”

This is the heart of the project, so it deserves its own section.

The problem: text-to-image models treat a character description as *inspiration*, not a *spec*, so they re-invent the character a little each time. Our fix is to remove all ambiguity:

1. **PARSE** emits a `locked_spec` per high character — one committed value per trait, hex colours, gaps invented-and-locked.

2. `charspec.serialize()` turns that into the *exact same text* every time.
3. `plan_v3.char_lock()` is the single place that produces character text, so **refs, generate, correct, and the judge all use the identical locked description** automatically.
4. The **judge compares the page to the reference *image*** (a real vision call), not just to text — so it catches drift the text can't describe.

Before this was wired in, the exact spec existed in code but was **orphaned** — the pipeline locked characters with vague free text (“two puffs *or* a headband”) and the judge compared against that. That single disconnect was the root cause of the “hair changes every page” rejection.

7. One rule-book, injected everywhere (`checklist_v3.py`)

We went through **every per-page client complaint** across two books (Bilbo & Obi, Ella) and collapsed them into one enforced rule set — so a fix is written once and propagates to every stage. It exports three renderings of the same rules:

- `t2i_block` → injected into **GENERATE**
- `i2i_block` → injected into the **CORRECT** edit
- `judge_block` → injected into the **vision judge**

Rules cover style, character consistency (incl. **PRO-1 consistent scale, GRD-1 grounded/no-floating, CRP-1 no legs-only crops, signature items always worn**), scene logic, one-unified-scene (no collages), verbatim text, front/back matter, uniform trim, and locked type. It even carries a **named banned-props list** (the hallucinated hotdog stand, safety vest, steam, parade...) so the model gets concrete negatives.

8. The version history — what we tried, and why each pivot happened

This is the “journey,” newest-relevant first. It's essentially a hunt for character consistency.

Version	What we tried	Why we moved on
v0.1–v1.4	First pipeline: parse manuscript, generate pages, place text on art (white-blur bloom, no card), assemble PDF; per-run versioning.	Worked as a pipeline, but characters weren't consistent and text sometimes seamed.

v1.5–v1.6	Locked character specs + a duo reference to stop two similar dogs collapsing into each other.	Helped, but refs “weren’t being utilised” — Obi still looked like a different dog per page.
v2.0	Compositing engine — draw each character once, reuse the exact pixels (sprites + compositor). Pixel-perfect at last.	Consistency solved, but composited characters read slightly “ pasted ” on the painted background. Shelved.
v2.1	Per-character LoRA on fal.ai — teach the model each character so it paints them naturally <i>and</i> consistently. Investigated Colab Pro for cheap training.	Judged the <i>right long-term</i> answer, but we paused it to first exhaust cheaper img2img options the client preferred.
v2.2–v2.3	Pivot to image-to-image correction : generate normally, then feed a drifted page + its reference to Gemini and edit only the wrong character. Found the in-place editor (google/gemini-3-pro-image) as the winning backend (earlier mask-inpaint and whole-page redraw both failed — one turned a page into a full-frame dragon).	Correction <i>helps</i> but is a “patch on a leak” — see §9. Plateaus at “pretty good.”
v3.0	Full TOON rebuild after the client rejected v5 with the wrong character model (both dogs are golden retrievers, distinguished only by hat colour — earlier versions had invented a beagle). Fresh refs from the real photo, coordinate scenes, correct + compose + book.	Big improvement and correct characters, but still text-to-image drift.

v3.1	Generic, manuscript-driven pipeline — <i>any</i> .docx runs end-to-end (built the Ella book from scratch for ~\$4). Vision-based text placement.	Generalized the tool; consistency still the open problem.
v3.2	Ballast house-style copyedit stage over the prose.	Additive quality stage.
v3.3	Turned all 32 pages of client complaints into the shared checklist injected into every image stage.	The rule-book that still governs everything.
v3.4	API hard-deadline so a hung connection can't freeze a run.	Reliability fix.
v4.0	Rebuild after the Ella rejection : single square pages (no spreads), locked_spec wired in (the orphaned spec finally connected), clean text in negative space (no border/clip/erratic size), title + dedication rendering. Deleted ~21 legacy modules.	This is the current <i>generate</i> engine. Consistency much better, but not perfect.
v4.3	Composition judging — a second critic for scale/grounding/crop, and signature items marked “always” so the judge stops flip-flopping and stripping bandanas.	Closed real gaps the identity-only judge missed.

v5.0	Layered compositing engine, revived + fixed — the shelved v2.0 idea plus the missing AI-harmonize step that removes the “pasted” look. Two dogs came out identical and the same size, reading as one painted scene.	The current default engine and the structural fix for the #1 issue.
-------------	---	--

9. Where it still fails — and why (the honest part for the expert)

Text-to-image (generate) drift is structural, not a quality bug. The model draws every page freehand, so coat shade, cap presence, and *size* wander. A fancier model draws *prettier* pages that are **just as inconsistent**.

The corrector is a patch on a leak. It only has a flat reference sheet, not “this exact character at this angle,” so it can match colours and items but can’t re-impose a face/body the model never truly learned. Fixing one thing can nudge another (colour↔pose↔face), so it can oscillate. Stacking *more* i2i passes doesn’t converge — each pass is another freehand draw that can undo the last fix.

The composite engine solves consistency but constrains art. Characters are exact, but they’re limited to the **poses in the atlas**, and the harmonize pass has to walk a line between “blend it in” and “don’t change the character.” Backgrounds are AI, so lighting between plate and sprite can disagree.

This is exactly where an image-gen expert can help. Our own analysis points at a **per-character LoRA** as the only approach that fixes the *cause* (teach the model the character so it’s right on the first draw, in any pose, painterly) rather than patching the symptom. We’d love a second opinion on: LoRA training set curation and count, base-model choice, whether an IP-Adapter / reference-conditioning route on Gemini or a Flux-LoRA route is the better bet, and how to get the composite harmonize pass to look fully hand-painted.

10. How to run it

From the repo root, using the project venv:

```
# full run, composite engine (default), on the Ella book:
.venv/bin/python -m pipeline.run_v3 --book books/ella-the-animal-shelter-and-you
# text-to-image engine instead:
.venv/bin/python -m pipeline.run_v3 --book <book> --engine generate
# resume from a stage, limit to pages:
.venv/bin/python -m pipeline.run_v3 --book <book> --from generate --only 4,11
```

```
# a side-by-side version (own output dir):  
V3_DIR=v3b .venv/bin/python -m pipeline.run_v3 --book <book>
```

Useful flags: `--from <stage>`, `--only <pages>`, `--no-copyedit`, `--no-correct`,
`--no-scene-pass`, `--no-harmonize`, `--docx <path>`.

Requirements: the project `.venv`, an `OPENROUTER_API_KEY` in `.env`, and the manuscript `.docx` under the book's `manuscript/` folder. A full book run costs a few dollars in API calls.

11. One-screen summary for the meeting

- **Goal:** manuscript `.docx` → print-ready illustrated PDF, automatically.
- **Models:** Claude Sonnet 4.5 (text + vision judging) and Gemini 3 Pro Image (drawing + in-place edits), both via OpenRouter.
- **TOON:** compact tabular data format the client asked for; we store JSON on disk for reliability and render TOON into prompts to save tokens.
- **Flow:** parse → copyedit → refs → [layout → generate → correct (*Engine A*) | sprites → layout → stage → render (*Engine B*)] → compose → book.
- **The one hard problem:** character consistency. Locked hex specs + reference-image judging + a shared rule-book got Engine A close; the **composite engine** (draw once, reuse pixels, AI-harmonize) makes it structural.
- **The proposed real fix:** per-character **LoRA** — teach the model the character so it's consistent *and* painterly from the first draw. This is what we want the expert's help to nail.